

Operating System Design Goals

Performance, Scalability, Reliability, Security, Portability and Interoperability

Prashant Jadav

Fundamentals of Operating System Design (FOSD)

Learning Objectives

By the end of this topic, students should be able to:

- Explain the major goals that influence operating system design.
- Analyze how OS design decisions affect performance and scalability.
- Distinguish reliability from availability and fault tolerance.
- Explain the role of security mechanisms in an operating system.
- Analyze portability and interoperability requirements.
- Identify trade-offs among different OS design goals.

Why Does an Operating System Need Design Goals?

An operating system is a **resource manager** and a **control program**.

Design goal

A property that guides architectural and implementation decisions so that the OS behaves effectively under its intended workload and environment.

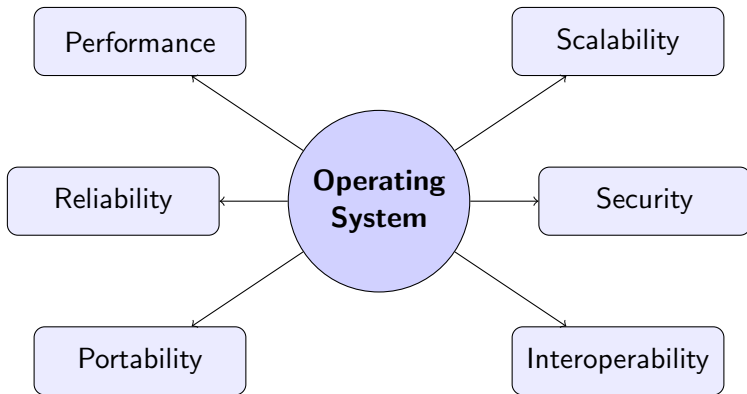
Resource management

- CPU
- Main memory
- Storage
- I/O devices
- Network resources

User/system requirements

- Fast response
- High availability
- Strong protection
- Hardware independence
- Cross-system communication

Major Operating System Design Goals



1. Performance

Definition

Performance is the ability of an operating system to complete work efficiently while providing acceptable response time.

Important performance measures

- **Response time:** Time between a request and the first response.
- **Turnaround time:** Time from job submission to completion.
- **Throughput:** Amount of work completed per unit time.
- **CPU utilization:** Percentage of time the CPU is busy.
- **I/O latency:** Time required to initiate/complete I/O.

Design challenge

Improving one metric may negatively affect another. For example, maximizing throughput does not always minimize response time.

How OS Design Affects Performance

Sources of overhead

- Context switching
- System calls
- Interrupt handling
- Synchronization
- Memory translation
- Process creation
- I/O operations

Performance techniques

- CPU scheduling
- Caching and buffering
- Efficient memory management
- DMA
- Multithreading
- Asynchronous I/O
- Efficient system-call paths

Example

A smaller context-switch overhead can improve throughput when a system runs many short-lived processes.

2. Scalability

Definition

Scalability is the ability of an operating system to maintain acceptable performance as system resources, workload, or the number of users increases.

Scalability dimensions

- More CPU cores
- Larger main memory
- More storage
- More concurrent processes/threads
- More users and network connections

Key idea

A scalable design should not require a proportional increase in overhead when the system size or workload grows.

Scalability: Example

Consider an OS running on a multicore processor.

Poor scalability

- Global lock protects a major kernel data structure.
- All cores compete for the same lock.
- More cores $\rightarrow$ more contention.
- Parallelism is limited.

Better scalability

- Fine-grained synchronization.
- Per-CPU data structures.
- Lock-free or wait-free techniques where appropriate.
- Greater parallel execution.

Important

Scalability is not simply “supporting more hardware.” The OS must exploit additional resources efficiently.

3. Reliability

Definition

Reliability is the ability of an operating system to perform its intended functions correctly and consistently over time.

Reliability mechanisms

- Error detection and handling
- Process isolation
- Memory protection
- File-system consistency
- Journaling
- Checkpointing and recovery
- Redundancy and fault-tolerant mechanisms

Reliability vs. Availability

Reliability focuses on correct operation over time, while **availability** focuses on whether the service is accessible when required.

Reliability and Fault Isolation

Goal: Prevent a failure in one component from bringing down the entire system.

Monolithic architecture

- Many OS services execute in kernel space.
- A kernel-level fault can have broad impact.
- High performance may be possible.

Microkernel architecture

- Minimal kernel provides core mechanisms.
- Many services execute outside the kernel.
- Better fault isolation can be achieved.
- Communication overhead may increase.

Design trade-off

Reliability and performance can pull architectural decisions in different directions.

4. Security

Definition

OS security protects system resources, processes, data, and users against unauthorized access, modification, disclosure, and disruption.

Core security objectives

- **Confidentiality:** Prevent unauthorized disclosure.
- **Integrity:** Prevent unauthorized modification.
- **Availability:** Keep authorized services accessible.

Typical mechanisms

- Authentication
- Authorization and access control
- User/kernel mode separation
- Memory protection
- Process isolation
- Secure boot
- Encryption

Security: Protection vs. Security

Protection

Controls how processes and users access resources *inside* the system.

Security

Protects the system against threats originating from users, software, networks, devices, and external attackers.

Mechanism	Primary purpose	Example
Authentication	Identify a subject	Password, biometric
Authorization	Decide permitted actions	File permissions
Isolation	Limit impact of a process	Virtual memory protection
Auditing	Record security-relevant events	System logs

5. Portability

Definition

Portability is the ability to move an operating system or its software components to different hardware platforms with minimal modification.

Factors affecting portability

- CPU instruction set
- Memory architecture
- Interrupt mechanisms
- Device interfaces
- Hardware-specific kernel code
- Compiler and language support

Example

An OS designed for both x86-64 and ARM64 must isolate architecture-dependent code from architecture-independent kernel components.

Improving OS Portability

Design techniques

- ① Separate hardware-dependent and hardware-independent code.
- ② Use hardware abstraction layers where appropriate.
- ③ Minimize architecture-specific assumptions.
- ④ Use standardized interfaces and APIs.
- ⑤ Use portable programming languages and build systems.

Hardware → Abstraction Layer → OS Services → Applications

Trade-off

Abstraction can improve portability but may introduce implementation complexity or runtime overhead.

6. Interoperability

Definition

Interoperability is the ability of an operating system and its applications to exchange data and services correctly with other systems, platforms, and software.

Examples

- Network communication using TCP/IP.
- File sharing using standard network protocols.
- Accessing storage using standard file-system formats.
- Exchanging data using common formats such as JSON or XML.
- Supporting standard APIs and system interfaces.

Key idea

Portability concerns **moving software across platforms**; interoperability concerns **working with other systems**.

Portability vs. Interoperability

	Portability	Interoperability
Main question	Can the software run on another platform?	Can the system communicate/work with another system?
Focus	Hardware/platform differences	System/software boundaries
Example	OS supporting x86 and ARM	Linux client communicating with a Windows server
Enablers	Abstraction, portable code	Protocols, standards, APIs, common formats

How the Design Goals Interact

OS design goals are **not independent**.

- More security checks can increase execution overhead.
- Strong isolation can improve reliability but may increase communication cost.
- Abstraction can improve portability but may add complexity.
- Fine-grained synchronization can improve scalability but complicates kernel design.
- Aggressive optimization can improve performance but make the system harder to maintain.

Central design principle

The objective is not to maximize every goal independently; the OS architecture must achieve an appropriate **balance for its target workload and environment**.

Design Goals Across Different Systems

System type	Perf.	Scale	Reliab.	Security	Port.	Interop.
Embedded OS	High	Medium	High	High	High	Medium
Mobile OS	High	High	High	High	High	High
Desktop OS	High	Medium	High	High	High	High
Server OS	High	Very High	Very High	Very High	High	Very High
Real-time OS	Predictable	Medium	Very High	High	High	Medium

Observation

The relative importance of a design goal depends on the system's intended use and workload.

Design Trade-off: A Practical Example

Scenario: Design an OS for a safety-critical industrial controller.

- **Performance:** Fast response is important.
- **Scalability:** May be less important than predictability.
- **Reliability:** Failure must be minimized.
- **Security:** Unauthorized control must be prevented.
- **Portability:** Useful when hardware platforms change.
- **Interoperability:** Important for integration with industrial networks.

Design question

Would you optimize the system for maximum raw throughput, or for predictable and dependable response? Justify your choice based on the application requirements.

Summary

Goal	Core objective
Performance	Execute work efficiently with low latency and high throughput.
Scalability	Continue to operate effectively as resources and workload grow.
Reliability	Perform intended functions correctly and consistently.
Security	Protect resources, data, and services from unauthorized actions.
Portability	Enable software to run across different hardware/platforms.
Interoperability	Enable effective cooperation and communication with other systems.

Key takeaway

OS design is a multi-objective engineering problem involving performance, scalability, reliability, security, portability, interoperability, and their associated trade-offs.

Higher-Order Thinking Questions

- ① A kernel performs very well on a single-core system but shows little performance improvement when CPU cores are increased. Which design goal is affected? Identify two possible causes.
- ② An OS uses strong process isolation, but communication between isolated services becomes expensive. Explain the trade-off between reliability and performance.
- ③ An OS works on both ARM64 and x86-64, but applications cannot easily exchange data with another OS. Which design goal is satisfied and which is not? Explain.
- ④ For a cloud server with thousands of concurrent users, which design goals require particular attention? Justify using workload characteristics.

Good OS design is not about maximizing one property.

It is about making informed architectural choices that balance:

Performance + Scalability + Reliability
Security + Portability + Interoperability

The appropriate balance depends on the target system and workload.